

C# Programming Lab

Mini Inventory System

Lab Title	Mini Inventory System
Language	C# (.NET Console Application)
Concepts Covered	Methods (parameters & return values), 1D Arrays

Objective

Build a console-based product inventory manager using parallel arrays and dedicated methods. Each method must have a single clear responsibility - no method should do more than one thing. This lab focuses on passing arrays to methods, returning values from methods, and keeping logic clean and organized.

Setup - Starter Variables

At the top of Main(), declare the following parallel arrays and a counter variable. Do NOT add any other arrays.

```
const int MAX = 10;                // maximum inventory slots

string[] productNames = new string[MAX];
double[] prices       = new double[MAX];
int[]    quantities   = new int[MAX];

int count = 0;                    // how many products are stored so far
```

Rule: All methods receive the arrays and count as parameters. No global variables.

Requirements

Part 1 - AddProduct()

Write a method with this signature:

```
static int AddProduct(string[] names, double[] prices, int[] quantities, int count)
```

The method must:

- Check if the inventory is full (count == MAX). **If full → print an error and return count unchanged.**
- Ask the user to enter: product name, price, quantity.
- Validate price > 0 and quantity > 0. **If invalid → print an error and return count unchanged.**
- Store the values in the next available index (index = count).

- Increment count and return the new count.

In **Main()**: `count = AddProduct(productNames, prices, quantities, count);`

Part 2 - DisplayAll()

Write a method with this signature:

```
static void DisplayAll(string[] names, double[] prices, int[] quantities, int count)
```

The method must:

- If `count == 0` → print "No products in inventory." and return.
- Print a formatted table with a header row and one row per product.
- Price must show 2 decimal places.
- If `price < 10.00` → append " Low Price" at the end of that row.

Expected output format:

#	Product	Price	Quantity	
1	Notebook	\$5.99	50	Low Price
2	Stapler	\$12.50	30	
3	Printer Paper	\$8.00	100	Low Price

Part 3 - SearchByName()

Write a method with this signature:

```
static int SearchByName(string[] names, int count, string target)
```

The method must:

- Loop through the names array.
- If a match is found (case-insensitive) → return the index.
- If no match → return -1.

In **Main()**, use the return value to either display the product's full details or print "Product not found."

Part 4 - GetTotalValue()

Write a method with this signature:

```
static double GetTotalValue(double[] prices, int[] quantities, int count)
```

The method must:

- Loop through the arrays and calculate: `total += price[i] * quantity[i]` for each product.
- Return the total inventory value as a double.

In **Main()**, print the result formatted to 2 decimal places:

```
Console.WriteLine($"Total Inventory Value: ${GetTotalValue(prices, quantities, count):N2}");
```

Part 5 - GetMostExpensive() & GetCheapest()

Write two methods:

```
static string GetMostExpensive(string[] names, double[] prices, int count)
static string GetCheapest      (string[] names, double[] prices, int count)
```

Each method must:

- If count == 0 → return "N/A".
- Find the index of the highest (or lowest) price.
- Return a formatted string: "Stapler (\$12.50)"

Part 6 - Main Menu Loop

In Main(), run a do-while loop that shows this menu until the user selects Exit:

```
INVENTORY MANAGER
1. Add Product
2. Display All Products
3. Search by Name
4. Show Total Value
5. Most & Least Expensive
6. Exit
```

- Each menu option calls the appropriate method.
- Invalid option → print an error and show the menu again.
- On Exit → print a goodbye message with the final product count.

Hints

Case-insensitive string comparison:

```
if (names[i].Equals(target, StringComparison.OrdinalIgnoreCase))
```

Safe number parsing:

```
if (!double.TryParse(Console.ReadLine(), out double price))
    Console.WriteLine("Invalid price!");
```

Formatted string return:

```
return $"{names[index]} ({prices[index]:N2})";
```

Finding the max index manually:

```
int maxIndex = 0;
for (int i = 1; i < count; i++)
    if (prices[i] > prices[maxIndex])
        maxIndex = i;
```

Bonus Challenge

Add a method:

```
static void UpdateQuantity(string[] names, int[] quantities, int count)
```

The method must ask the user for a product name, search for it using `SearchByName()`, and if found, ask for a new quantity (must be ≥ 0) and update it. If not found, print an error.

Add a "Restock Alert" - after every `DisplayAll()` call, print the names of all products whose quantity is less than 10.

Good luck!